essay · 2026-10-02

what i learned rebuilding pi audit loop

Adrian Tame Jacobo, writing for Plicara Labs · 2026-10-02 ·

Pi Audit Loop began as a small way to alternate code review and simplification, a loop I had found I quite liked when working with agentic systems for coding. Review the code, simplify what can be simplified, then look again. A state machine kept track of the phases and the stopping conditions. It found real problems, but testing it exposed a million real flaws in the design, primarily: when a review found a behaviour bug, the next phase was supposed to preserve behaviour. The tool had no legitimate route for fixing the thing it had just found!

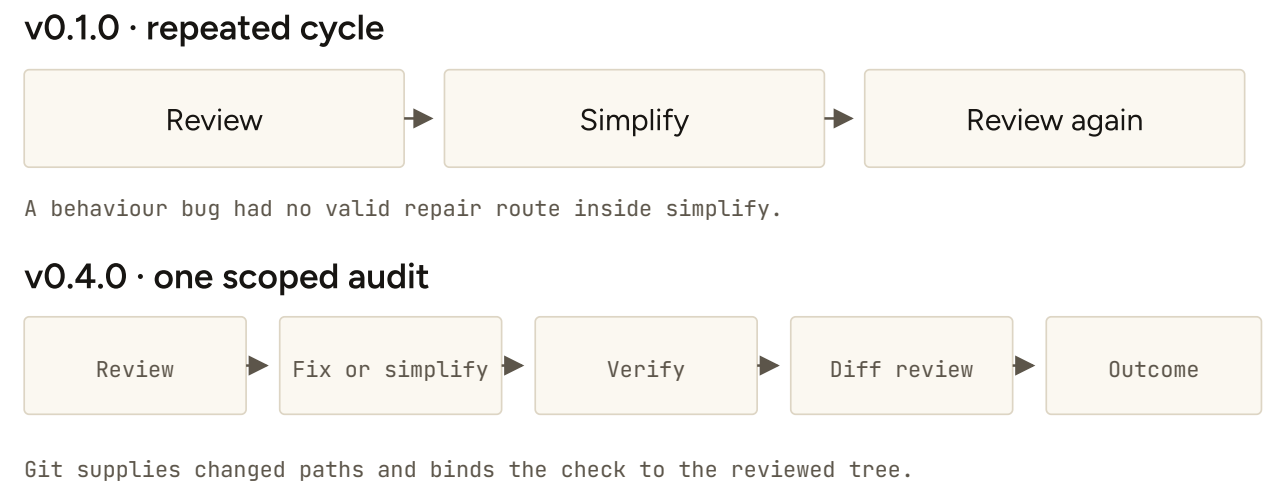
The first experiments also showed how easily a useful result can be mistaken for a guarantee. I wrote an instruction the models never read, briefly measured code that an earlier run had already changed, and watched a model make a test suite pass while introducing another bug, in that same suite. Those results led to a different package: one focused review, one selected fix or simplification, verification tied to the repository state, and a review of the resulting diff.

I wrote recently about which responsibilities belong in the harness around a model. This was a small, practical way to explore that question. What can we put into ordinary code, and what still depends on the model making a good judgement? The September 2026 fixture and real-project runs below tested v0.1.0 during its development, which I do not recommend anyone try. The later v0.4.0 sweep is a separate set of development runs of a different workflow really.

the original loop

Pi Audit Loop is a package for the pi coding agent. The original v0.1.0 package gave the agent tools for starting an audit, recording a review, recording a simplification, checking the current state and stopping. Behind those tools was a state machine: code that recorded the current phase and decided which transition was allowed next.[1]

FIGURE 1. FROM A REPEATING LOOP TO ONE CHECKED CHANGE



The original loop alternated review and simplification. The current graph has an explicit fix route, runs a configured check and reviews the resulting diff. This shows the designed transitions, not measured review quality.

In that first design, the review asked what needed attention. The simplification pass could remove duplication or make code easier to follow, provided it preserved behaviour. A changed pass returned to review unless the round budget was exceeded. A review with no findings could finish the loop; a pass with nothing to simplify could finish it too. Some possible endings can occur from this pattern, listed below.

recorded ending	what i can conclude
`review_clean`	The model reported no actionable findings. The configured test gate, when present, also passed.
`nothing_left`	The model reported no further simplification. Bugs may still be open.
`budget_exhausted`	The loop stopped at its budget condition. The remaining findings need attention.
`stopped`	Somebody explicitly stopped the loop.

Those distinctions mattered. Finding a bug and stopping because fixing it would change behaviour was an honest outcome for the original tool. Calling every termination a successful repair would have lost exactly the information I wanted to keep. The table describes the v0.1.0 record, because I want to describe the failure here, it is what I found interesting.

The phase logic was separate from the pi adapter, which made it testable without running a model. I also bundled existing review and simplification skills, adapted for these tools, with their source revisions and licences recorded. The review guidance came from Anthropic's knowledge-work plugins, and the simplification guidance from Addy Osmani's agent-skills repository.[2] That gave me a useful starting point for what happened inside each phase. I still had to work out whether those instructions actually reached the model.

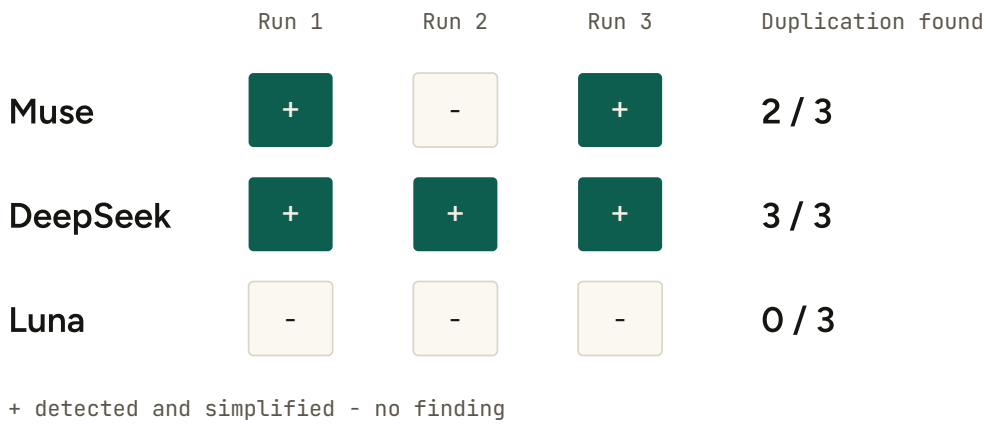
give the experiment an answer

Before running this over larger projects, I made small Python projects with known answers. I wanted to be able to say what the loop should find before seeing what it told me.

fixture	what i put in it	what i wanted to observe
A: a hidden bug	A zero discount falls through to the default discount. The existing tests pass.	Does review find a defect the suite misses?
B: duplication	Three copies of the same aggregation logic, differing in their grouping key.	Does the loop find and remove the duplication?
C: clean code	A small implementation with guards and tests, and no planted defect.	Can it leave the code alone?
D: failing tests	Asking for the last zero items returns the whole list.	Will it respect the boundary between simplification and repair?

The first attempt at the duplication experiment went wrong though. I ran successive models against the same directory. The first model removed the duplication, so the later models saw code that had already been cleaned up. What looked like a difference between models was partly a difference in their inputs. I discarded those 3 runs and reset the fixtures before the replacement runs. That was a useful reminder to inspect the thing being measured, even when the harness itself appears to be working.

FIGURE 2. SAME DUPLICATION, SEPARATE RUNS



Each square is one reset-fixture run, in order for that model. Counts describe this fixture only; three observations do not establish a general model ranking.

After resetting the fixtures, all three models found the hidden bug in A and returned no findings on C. Each of those was one run per model. B had repeated runs: DeepSeek found the duplication in 3 of 3, Muse in 2 of 3, and Luna in 0 of 3. The chart shows the individual outcomes because that is about as much precision as these measurements support.

I would not turn that into a general ranking of the models. I tested tiny projects through one provider, `opencode-go` (incredible value by the way on that product), and three repeats do not establish a stable success rate. It did establish something useful for this project: the same procedure could produce different judgements on the same code. Getting the tools called in the right order did not make the duplication equally visible to every model.

Across the full record there were 26 fixture runs, 2 runs on real repositories and 2 deliberate guard probes. Excluding the probes, all 28 runs reached a recorded terminal state and there were 0 rejected audit calls. The machinery was usable by the models I tried, even if it was not very good, as will be detailed in the next section. There was no prompt-only control group, so this does not tell us how much the loop improved on asking for a review directly.[3]

the tests passed, and the change was wrong

The failing-test fixture gave me the clearest example of what the state machine could not check. Its function returned `items[-n:]`. In Python, negative zero is still zero, so asking for the last zero

elements returned everything.

In one run, DeepSeek replaced the slice with `items[len(items) - n:]`, keeping the surrounding negative-input guard. That fixed the zero case. It also introduced a different failure when the requested tail was longer than the list.

FIGURE 3. A PASSING SUITE LEAVES A HOLE

`Candidate: items[len(items) - n:]`

Input list: [1, 2, 3]. Requested tail length n:



`n = 4 → [3]` `n = 5 → [2, 3]`

Both should return [1, 2, 3]. At n = 6 the candidate works again.

The candidate passes all 4 existing tests. For nonnegative n, the regression lies strictly between the list length and twice that length. The oversized test uses length 2, n = 5, outside that interval. This figure is a deterministic replay, not a new model run.

For `[1, 2, 3]`, asking for the last five items should return the whole list. The replacement returns `[2, 3]`. The existing oversized-input test used a different case, a two-element list with five requested, where the replacement happens to work. Replaying the candidate against the fixture's 4 tests still produces a pass. Sweeping nearby inputs exposes the regression immediately.

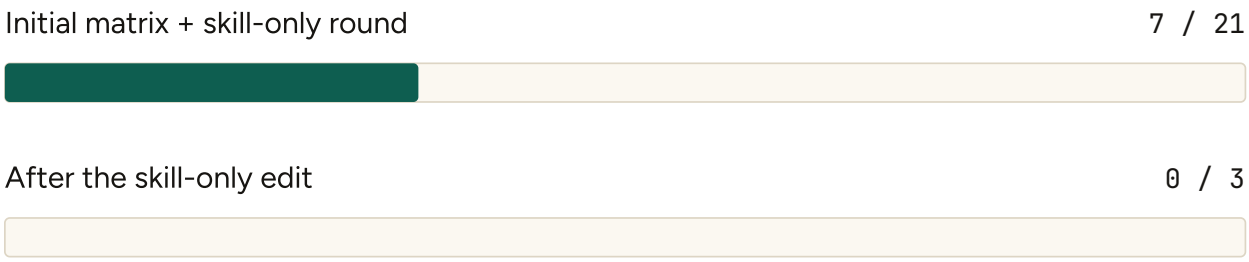
There were two problems here. The model changed behaviour during a phase intended to preserve it, and the tests did not catch the new behaviour. The state machine accepted a report that files had changed and later accepted a clean review. It had no representation of what the edit meant.

I think that distinction matters when describing a tool like this. Running tests gives us evidence about the cases those tests cover. The loop can make some parts of that process explicit, but the meaning of the change still needs review. My response was to ask the next review to examine the simplification's diff, as well as running the suite.

it also put in a hidden fix

The first place I wrote that instruction was the bundled review skill. That seemed reasonable: it was guidance about how to review. Then I checked the validation runs and found that none had opened the skill (!).

FIGURE 4. DID THE FULL SKILL ENTER THE CONVERSATION?



Share of sessions with a read-tool call targeting SKILL.md. The second row is a subset of the first. These are reads, not measurements of comprehension or compliance.

In the initial matrix plus the next guidance round, 7 of 21 sessions called the read tool on a `SKILL.md`. In the 3 runs immediately after my skill edit, 0 did. The full instructions were loaded on demand, which is how the Agent Skills specification describes the format: the agent sees identifying information first and loads the body when it activates the skill.

I moved the short rule into the tool description and the message returned when a simplification sends the loop back to review. That puts it at the point where the model needs to act on it. The fuller explanation still lives in the skill.

failing-test fixture	original guidance	rule added to skill	rule added to tool message and description
Muse	Left bug open	Left bug open	Left bug open
DeepSeek	Changed behaviour; introduced regression	Left bug open	Changed behaviour
Luna	Changed behaviour	Changed behaviour	Left bug open

Each cell is one run. Luna left the bug open in the last configuration, but DeepSeek changed behaviour there after leaving it alone in the previous one. I established that the instruction was delivered through the new channel. I did not establish that it reliably changed the outcome. I still think moving it was the right change, because an instruction has to reach the model before we can reasonably ask whether it helps.

the useful results were on real code

I also ran the original loop on fresh copies of Loadstar, a Go project of mine, and Simon Willison's sqlite-utils. Both started with passing suites. These were runs during v0.1.0 development, before its test gate was added, and both used the same DeepSeek model.

FIGURE 5. SAME MODEL, DIFFERENT HANDLING OF FINDINGS IN V0.1.0

loadstar

`6 findings`

Findings acted on, including repairs

`review_clean`

Behaviour changed during simplify

sqlite-utils

`3 findings`

Repeated logic extracted; bugs left open

`nothing_left`

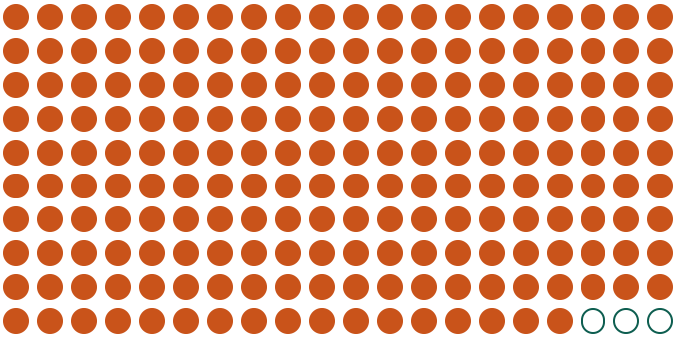
Findings still need a repair decision

ONE LOADSTAR CONCURRENCY PROBE

197 / 200

calls failed with

`no such table: jobs`



Both original-loop development runs used DeepSeek on fresh project copies with passing baseline suites. The endings are recorded states, not quality scores. Each dot below is one concurrent call in the Loadstar probe; filled dots failed, outlined dots succeeded.

Loadstar produced 6 reported findings. The most concrete was an in-memory SQLite problem under concurrency: separate pooled connections could see separate databases, so a connection could arrive at a database with no jobs table. A probe in the run recorded 197 failures out of 200 concurrent calls. That is a result from one reproduction, not an estimated production failure rate.

The model also found a cancellation path that could leave a job stranded after a store error, missing checks for errors during row iteration, and a retention task that skipped independent work after a failure. It acted on the findings and added regression tests. That was useful work. It also included repairs during the simplification phase, so the clean ending did not establish that the phase had stayed within its intended scope.[4]

In sqlite-utils, the same model behaved differently. It left 3 findings open, including a `--limit 0` path that returned every row and a list-insertion path that shifted values when a hash ID was requested. It made separate simplifications by extracting repeated logic for primary keys, row output and journal mode. The recorded suite still showed 1485 passed and 19 skipped, and the loop ended with `nothing_left`.

That contrast is why I regard the project as useful while staying quite specific about its guarantees. The findings gave me concrete things to investigate, and the simplifications could be inspected as diffs. The phase name alone did not tell me whether the model had preserved behaviour. Also, the Loadstar finding list included a misleading comment, so I would describe the combined result as findings rather than counting every item as a behavioural bug.

turning one instruction into a check

The final change before release was to enforce part of the test requirement. Until then, running tests before reporting clean was an instruction. In the 18 clean verdicts in the non-probe record, the logs already contained a prior successful tool result for an invocation containing the configured test command. That is encouraging, but it measures what those runs did. It does not show that the earlier extension would have refused a verdict without it.

With a test command configured, v0.1.0 refused clean until it had observed a matching bash invocation finish successfully. A reported simplification reset that verification state. I checked the refusal with deliberate probes, including a loop whose command was `echo TESTS`; calling clean before running it returned `audit_rejected`.[5]

part of the process	what v0.1.0 checked	what remained outside that check
Phase order	Whether the requested transition is legal	Whether the agent keeps working rather than abandoning the session
Change report	Whether `changed` agrees with the presence of a file list	Whether that list matches the actual edits
Test verification	A matching bash invocation completed successfully, when a command is configured	Whether the command really exercised the intended tests, and whether those tests are sufficient
Simplification	Records the pass and directs the next review	Whether the change preserved behaviour

There are implementation limits here too. The command is optional, matching uses a substring of the bash command text, and the reset follows a reported simplification rather than independently watching every file edit. `echo TESTS` is enough to illustrate why a successful command is not automatically a meaningful test. I would treat this as a useful check within a cooperative workflow, with the command and resulting diff still needing inspection.

The package also needed the less interesting work that makes a small tool usable: correcting the runtime dependency declaration, including the vendored licence, and fixing a status field that counted reviewed files as changed files. That last bug surfaced while using the tool. A clean fixture gave me a particularly good way to notice that the status display was claiming changes where none should have existed.

give a finding somewhere to go

The next design decision followed from the contradiction in the original loop. A behaviour bug needs a fix, while a simplification should preserve behaviour. Version 0.2.0 replaced the repeated cycle with one bounded path: start, initial review, one selected `fix` or `simplify` change, verification, final diff review, then an explicit outcome. A clean initial review goes straight to verification. A finding left unchanged ends as `open_findings`; another finding starts another audit.[6]

I considered making online research and test-driven development mandatory phases. That would add work even when the relevant behaviour is already clear or the change is only a simplification.

The current review guidance asks the agent to establish expected behaviour from the task and repository, then consult external documentation when an API, security claim or standard is uncertain. For a behaviour fix, it asks for a failing regression before the implementation change. Those are instructions to the agent; the machine does not prove that the test failed first or that the research was sufficient.[7]

The initial and final reviews now require a short `basis`: what was examined, where a finding lies, and what evidence supports the verdict. That makes the judgement inspectable in `audit_loop_status` and the session record. The package also restores the last valid audit state when a Pi session resumes on its active branch. A recorded basis is still a claim, but losing the reason for a verdict was an avoidable gap in the first dogfood round.[7]

check the revision that actually passed

The first version of the new workflow still let the agent supply its own changed-file list. A passing check could also be followed by another edit before final review. Version 0.4.0 made Git the evidence source for changed paths and tied verification to a fingerprint of the checkout. A new audit requires a Git repository. It captures a baseline that includes staged, unstaged and untracked files, including edits that were already present when the audit began. `audit_change` compares the current tree with that baseline; the agent no longer supplies `changed` or `files`.[6]

`audit_verify` runs the configured command in the project directory and records its exit status. It refuses to treat the run as passed when the tree has changed since the recorded change or clean review, or when the command itself changes the tree. The final review checks that the tree still matches the verified fingerprint. The command is optional, but a run without one ends `unverified`. This gives a passing check a specific revision to describe. It cannot tell whether the command is a meaningful test, whether the finding is correct, or whether a simplification preserved behaviour. Git-ignored files are outside the fingerprint.[6]

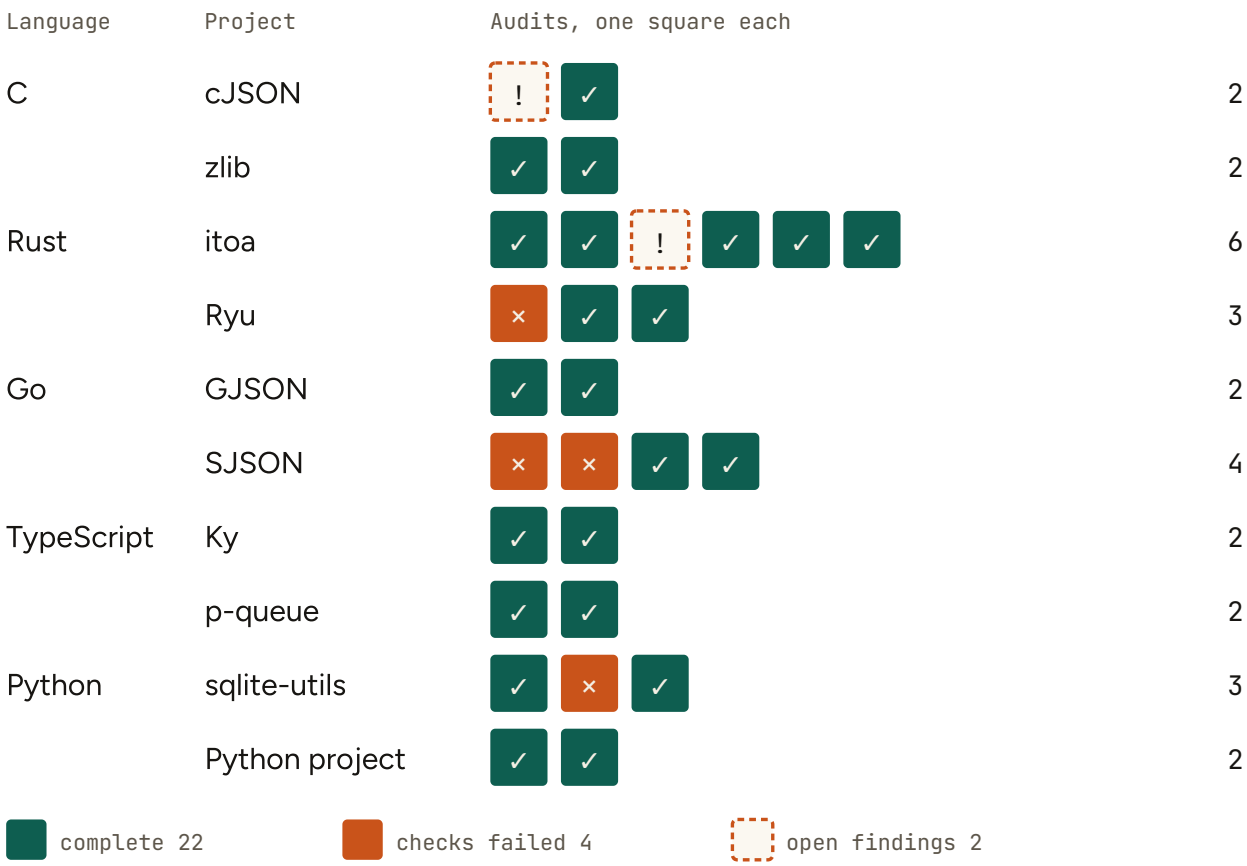
current ending	what the record says
`complete`	A clean review and a configured check passed on the checked tree. After a change, the final diff review was also clean.
`open_findings`	A finding remained after no change or the final review.
`checks_failed`	The check failed or the repository changed across a verification boundary.
`unverified`	No test command was configured.

current ending	what the record says
`stopped`	Somebody explicitly stopped the audit.

what happened on the new workflow

During v0.4.0 development, I tested the new workflow with Pi in ten disposable checkouts: two projects each in C, Rust, Go, TypeScript and Python. By my count from the session logs, the sweep recorded 28 audit starts and 28 explicit outcomes: 22 `complete`, four `checks_failed` and two `open_findings`. The per-project notes linked below describe each scenario and the independent test run that followed it. Some failed or open audits were followed by new, successful audits of the same scope; those earlier outcomes remain in the count. These are workflow outcomes, not 22 independently correct patches or a success rate for the package.[8]

FIGURE 6. TWENTY-EIGHT AUDITS ACROSS TEN PROJECTS



Each square is one audit in the v0.4.0 sweep. A failed or open audit was followed by a new audit of the same scope, and the earlier outcome stays in the count. The order of audits inside a project is not recorded. The split by project is read from the package notes and the totals reconcile with them; the raw session logs are not published.

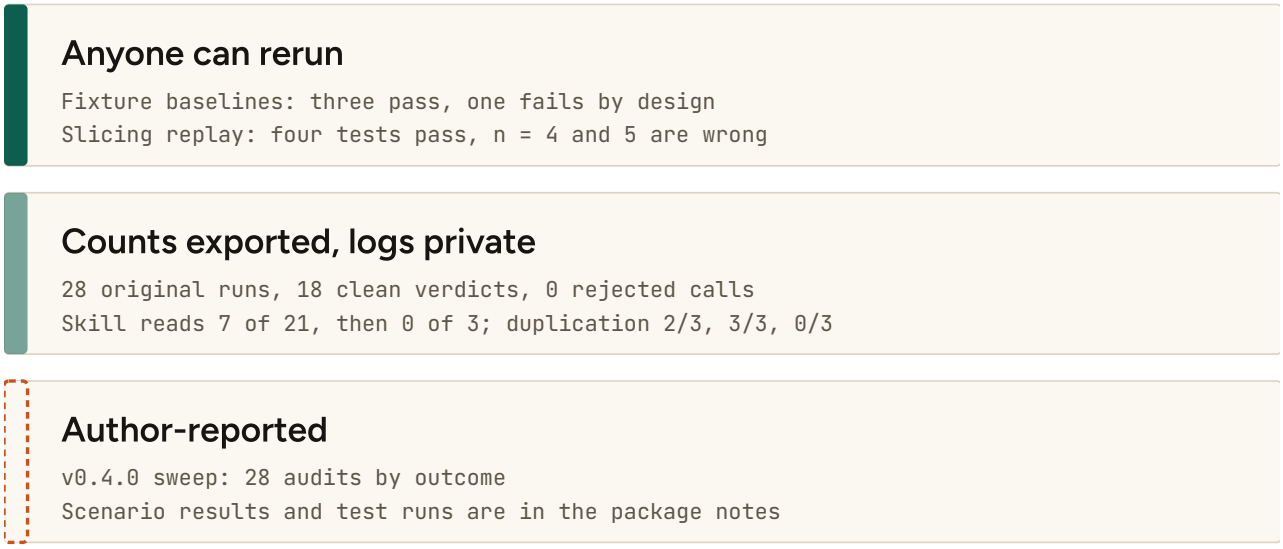
In Ryu, the system added a test probe after a clean review; in SJSON, it edited source after recording the change; in sqlite-utils, it edited after verification. The package recorded failures before those trees could be reported as checked. A GJSON run resumed after a provider timeout with the saved phase restored. In a separate live smoke, Pi fixed a seeded Rust sign regression, `audit_change` reported `src/lib.rs` from Git, and an independent offline Cargo run passed. These runs exercised the boundaries I had added; they did not measure review accuracy across languages.[8]

The sweep also showed where judgement remains. A WAL helper extraction in sqlite-utils passed the suite and the final review, yet its net clarity gain was debatable because it replaced two short

methods with an extra layer. Later guidance asked for a concrete clarity gain, and a focused retest left that code alone. One retest cannot establish that the wording reliably prevents needless edits. The extension can require a final diff review, but it cannot make that review perceptive.[8]

For another evaluation, I would keep the known-answer fixtures, add a direct-review baseline and repeat the harder cases. The original matrix and the later sweep cannot establish a productivity gain, a cost advantage or a general improvement in review quality. They do show why the redesign was necessary: the original loop had nowhere legitimate to put a behaviour fix, and the first one-pass version could call a changed tree verified. The current package gives both cases an explicit path and an observable check.

FIGURE 7. WHAT EACH NUMBER RESTS ON



The article's evidence by how far a reader can check it. The top band is deterministic and ships with the public bundle. The middle band is recomputed from logs only the author holds. The bottom band is the author's own count from the sweep.

I still use the same question to judge this tool: what evidence supports the result it reports? The answer is now clearer about phase, change, command and revision. The quality of the review and the adequacy of the tests still have to be examined by someone reading the work.

sources and measurement notes

The original-run model identifiers are `opencode-go/muse-spark-1.3-contributor`, `opencode-go/deepseek-v4.1-flash` and `opencode-go/gpt-5.6-luna`; those runs were collected on 13 September 2026. The slicing figure is a deterministic replay performed while preparing the article, not an additional model run. The later dogfood used `opencode-go/mimo-v2.6-flash` in disposable checkouts on 27–28 September 2026. The two sets of runs tested different package versions and tasks. No cost, latency or productivity comparison was collected for this article.

figure and claim trace

figure or claim	value or source	how it was checked
Original architecture and endings	v0.1.0 phase machine and adapter	Read against source; no claim of semantic enforcement
Duplication runs	Muse 2/3; DeepSeek 3/3; Luna 0/3	Recomputed from first review calls in the reset matrix; consistent with recorded outcomes
Slicing regression	4 existing tests pass; wrong outputs shown in figure	Candidate replayed against the fixture tests; inputs swept directly
Skill reads	7/21; 0/3 after the edit	Recomputed read-tool calls in the named cohorts
Real-repository comparison	6 and 3 reported findings; different endings	Session verdicts and terminal states, with changes described in the recorded results
Concurrency probe	197/200 calls failed	Original probe code and its tool result; not rerun as a performance estimate
sqlite-utils suite	1485 passed; 19 skipped	Original pytest tool results inspected
Full run record	28 non-probe runs; 18 clean verdicts; 0 rejected audit calls	Recomputed from JSONL; all clean calls had a prior successful matching-command result

figure or claim	value or source	how it was checked
Current graph and Git checks	v0.4.0 at `40061bc`	Read phase machine, adapter, repository snapshot code and package decisions
Later sweep	10 checkouts in five languages; 28 starts; 22 `complete`, four `checks_failed`, two `open_findings`	Recounted starts and terminal outcomes from twelve Pi session logs that are not published; ten-checkout split and scenario details checked against the package's 28 September dogfood note
Resume and clarity examples	GJSON provider timeout, sqlite-utils WAL retest	Package's dated notes; scenario-level observations, not a controlled model comparison

The evidence bundle holds the fixtures, scorer, protocol, per-session counts and the sweep's outcomes by project. It was built from an allowlist and scanned for personal paths and identifiers before release. The sweep's raw session logs are not published, so its aggregate counts are the author's own.

---

1. Pi Audit Loop source at v0.1.0, particularly the phase machine and extension adapter. Architecture and enforcement descriptions were checked against the local source, which matches the release for these files. ↩

2. Vendored sources and adaptations. Review skill: `anthropics/knowledge-work-plugins` at `a6d8653261a4`; simplification skill: `addyosmani/agent-skills` at `be4e44a9fbc5`. ↩

3. Public evidence bundle: `results/matrix.md` and the per-session export in `results/evidence.json`, with the protocol, fixtures and scorer under `scripts/matrix/`. The export was recomputed from the original JSONL logs with `scripts/verify_evidence.py`; contaminated sessions are excluded. The earlier results file contains interim denominators, so totals here use the full exported record. The fixture chart uses only the original reset matrix, excluding later guidance and gate checks. ↩

4. Public evidence bundle: `results/dogfood.md`, checked against the original session tool results for the concurrency probe and sqlite-utils suite. These findings describe the copies audited at the time; they are not claims about the current upstream projects. ↩

5. Verification-gate implementation. The pre-gate runs and post-gate checks are distinguished in the bundle's `results/evidence.json`. A prior successful command result is an observation, not a claim that earlier runs were protected by this gate. ↩

6. Pi Audit Loop v0.4.0 source, especially the phase machine, Git evidence and Pi adapter. The local package checkout was clean at commit `40061bc`, the v0.4.0 tag, when the article was written. ↩↩↩

7. The package's dated one-pass graph, review evidence and resume, and Git evidence decisions record the alternatives and reasons. These decisions explain the intended design; the linked source shows what shipped. ↩↩

8. The package's v0.2.0 dogfood, v0.3.0 long-horizon dogfood and v0.4.0 cross-language dogfood give scenario-level results and producing commands. I recounted the v0.4.0 starts and terminal outcomes from twelve Pi session logs, including the GJSON resume log. Those logs and the disposable checkouts are not published, so the aggregate counts are author-reported. `sweep-outcomes.json` in the evidence bundle lists the 28 outcomes by project, and the scenario-level claims rest on the dated notes and the test results recorded in them. ↩↩↩